

Лабораторная работа №3

Анализ безопасности кода

Части: 3.1 — анализ безопасности кода разработанного приложения; 3.2 — методика оценки безопасности кода

Приложение: `pziis-memvault` — хранилище записей целиком в оперативной памяти, без персистентности

Стек: Python 3.12, интерактивное меню в терминале, контейнер

Состояние: приложение не реализовано; выполнены разбор темы, 11 подготовительных экспериментов и план

Этот документ: объяснение каждой темы простым языком, результаты экспериментов, что предстоит, ответы на вопросы

1. О чём эта работа простыми словами	2
2. Что такое безопасный код	3
3. Валидация ввода и аварийные ситуации	3
4. Что такое дамп памяти и чем он опасен	5
5. Почему секрет остаётся в памяти	5
6. Как секрет утекает на диск	7
7. Средства защиты памяти	7
8. Профилирование: память и процессорное время	9
9. Результаты подготовительных экспериментов	9
10. Что предстоит реализовать	11
11. Часть 3.2: методика оценки	13
12. Вопросы и ответы	14
13. Итог	16

1. О чём эта работа простыми словами

Первые две работы смотрели на систему снаружи. Третья открывает крышку и смотрит внутрь процесса.

Методичка формулирует этот сдвиг сама: «в предыдущих лабораторных работах безопасность программных систем анализировалась по принципу **чёрного ящика**». Третья работа — это **белый ящик**: изучается исходный код и то, что происходит в оперативной памяти.

	ЛР1	ЛР2	ЛР3
Объект	права на объекты ОС и СУБД	приложение снаружи	код и содержимое памяти
Где данные	файловая система, база	база данных	только оперативная память
Ключевой вопрос	кому что разрешено	устоит ли против атак	что остаётся в памяти после работы с секретом
Роль исполнителя	настройщик	проектировщик	исследователь кода

СВЯЗЬ С ПРЕДЫДУЩЕЙ РАБОТОЙ

Третья работа — не продолжение второй по коду, но прямое продолжение по смыслу. В анализе защищённости второй работы остаточный риск был сформулирован так: «открытый текст неизбежно присутствует в памяти процесса во время обработки запроса; Python не позволяет надёжно затирать строки».

Третья работа берёт именно этот риск и доводит его до измеримого результата. На защите это стоит проговорить: работа не начинается с нуля, а закрывает зафиксированный ранее пробел.

Важное отличие от второй работы: базы данных нет

Задание требует создавать, обновлять и удалять данные **в оперативной памяти**. Никакой персистентности — ни базы, ни файлов. Это не упрощение, а смена предмета изучения: интересен не доступ к данным, а их **жизненный цикл внутри процесса**. Появился секрет в памяти — что с ним происходило — что от него осталось после удаления.

2. Что такое безопасный код

Методичка даёт определение: **безопасный код** — это код, позволяющий минимизировать риски нарушения конфиденциальности и доступности данных при работе программного обеспечения в любых ситуациях.

Ключевые слова здесь — «в любых ситуациях». Дальше методичка перечисляет, какие именно ситуации надо учитывать:

Ситуация	Что может пойти не так
Невалидные данные: длинные строки, некорректные значения, неверные типы	программа падает с подробной ошибкой, зависает, потребляет всю память или молча портит данные
Аварийные ситуации: сбой программы или оборудования	при падении создаётся дамп памяти, и конфиденциальные данные оказываются на диске в открытом виде
Большие нагрузки: сетевые, процессорные, по памяти	отказ в обслуживании; данные вытесняются в раздел подкачки, то есть попадают на диск

И называет два требования к тому, как писать код:

1. писать так, чтобы **конфиденциальные данные из оперативной памяти не могли попасть на жёсткий диск или быть переданными по сети** — через кэш или дамп;
2. корректно настраивать окружение и условия запуска приложения.

КАК ЭТО ЗВУЧИТ НА ЧЕЛОВЕЧЕСКОМ ЯЗЫКЕ

Вторая работа отвечала на вопрос «что будет, если украдут базу данных». Третья отвечает на вопрос «что будет, если снимут снимок оперативной памяти работающей программы». И оказывается, что второе сложнее: базу можно зашифровать целиком, а в памяти данные обязаны когда-то побывать в открытом виде — иначе с ними невозможно работать.

3. Валидация ввода и аварийные ситуации

3.1. Валидация ввода

Валидация — проверка входных данных до того, как они попадут в логику программы. Методичка предлагает решать проблему невалидных данных **валидационными сообщениями**: программа не падает, а объясняет пользователю, что именно не так.

Что нужно проверять и почему:

Проверка	Зачем
----------	-------

Длина строки	строка на миллион символов может исчерпать память или подвесить обработку
Тип значения	там, где ожидается число, может прийти текст — и программа упадёт на арифметике
Диапазон числа	отрицательный или огромный индекс приводит к обращению за границы структуры
Управляющие символы и нулевой байт	ломают вывод, журналы и обработку строк
Нормализация Unicode	одна и та же буква записывается разными кодами, поэтому сравнение без нормализации даёт неверный результат

Правило. Проверять надо по **белому списку**: перечислять допустимое, а не пытаться перечислить недопустимое. Недопустимого всегда больше, чем можно предусмотреть.

3.2. Поведение в аварийной ситуации

Когда программа падает, происходят три вещи, каждая из которых — потенциальная утечка:

- **Печатается трассировка** — а в ней значения переменных, включая секреты. Защита: свой обработчик исключений, который не выводит содержимое переменных наружу.
- **Создаётся дамп памяти** (core dump) — полный снимок памяти процесса на диск. Защита: запретить создание дампов.
- **Пишется журнал** — и если в него попадают данные пользователя, секрет оказывается в текстовом файле. Защита: писать в журнал только идентификаторы, никогда содержимое.

4. Что такое дамп памяти и чем он опасен

4.1. Что это

Дамп памяти — файл, содержащий побайтовую копию оперативной памяти процесса: все его переменные, буферы, ключи, пароли — всё, что в этот момент было в памяти.

Дампы создаются в трёх случаях:

Когда	Кто создаёт
Программа упала	ядро операционной системы автоматически, если разрешено
Программа работает	отладчик или администратор командой — процесс продолжает работать
Целиком вся память машины	средства криминалистического анализа, гибернация

4.2. Чем опасен

Дамп кладётся на диск обычным файлом. Дальше он может попасть в резервную копию, уехать в систему сбора отчётов об ошибках, остаться на диске после списания оборудования. Всё это время секреты в нём лежат открытым текстом.

В перечне CWE это отдельная слабость: **CWE-528** — доступность core-дампа посторонним. А сама причина — **CWE-316**, открытое хранение конфиденциальных данных в памяти.

4.3. Как анализируют дамп

Самый простой способ — поиск текста. Утилита выбирает из двоичного файла последовательности печатных символов, и в них ищется нужное. Если пароль был в памяти открытым текстом, он найдётся обычным поиском по подстроке.

Более точный способ, который и планируется в работе, — не просто найти вхождение, а определить, **в какой области памяти** оно лежит: в куче, в стеке, в отображённом файле. Карта областей процесса доступна через специальный файл в `/proc`, и по адресу вхождения можно сказать, что именно его удерживает.

5. Почему секрет остаётся в памяти

Это центральная тема работы, и она устроена непривычно: удалить переменную в языке высокого уровня не значит стереть данные из памяти.

5.1. Изменяемые и неизменяемые объекты

В Python есть два вида объектов, и разница между ними здесь определяющая:

Вид	Примеры	Можно ли затереть содержимое
-----	---------	------------------------------

Неизменяемые	<code>str</code> (строка), <code>bytes</code> , числа	Нет. Объект нельзя изменить на месте — только создать новый. Старый остаётся в памяти, пока его не перезапишет что-то другое
Изменяемые	<code>bytearray</code> , массивы	Да. Содержимое можно перезаписать нулями на том же месте

5.2. Почему `del` не помогает

Оператор `del` и сборщик мусора не стирают данные. Они лишь помечают память как свободную, чтобы её можно было выдать под другие объекты. **Содержимое остаётся лежать до момента, когда его случайно перезапишут.** Между освобождением и перезаписью может пройти сколько угодно времени — и в этот промежуток дамп покажет секрет.

Это подтверждено экспериментом: после `del` и явного вызова сборщика мусора секрет находился в дампе столько же раз, сколько и до удаления.

5.3. Правильный способ: затирание изменяемого буфера

Секрет нужно с самого начала держать в изменяемом буфере (`bytearray`), а после использования **перезаписать его нулями**. Перезапись выполняется низкоуровневой функцией `memset`, которая работает прямо с памятью буфера.

Эксперимент показал: затирание действительно убирает ровно ту копию, которая была затёрта.

ГЛАВНАЯ НАХОДКА: ДЕЛО НЕ В ХРАНЕНИИ, А ВО ВВОДЕ

Затереть своё хранилище просто. Проблема в другом: пока секрет добирается от клавиатуры до вашего буфера, по пути создаются **промежуточные копии** — и все они неизменяемые.

Стандартное чтение строки возвращает неизменяемые `bytes`. Преобразование в строку создаёт ещё одну копию. Разбиение на части — ещё по копии на каждую часть. Ни одну из них затереть невозможно. В эксперименте все четыре варианта хранения дали одинаковый результат именно поэтому: утечку обеспечивал не способ хранения, а путь ввода.

Вывод, который и станет ядром работы: **секрет должен жить только в изменяемых буферах от начала и до конца**. Читать нужно сразу в заранее выделенный буфер, разбирать — не создавая копий, а через «окно» в тот же буфер.

Когда этот принцип соблюлён, результат достигается: в эксперименте строгий путь на изменяемых буферах дал **ноль вхождений** секрета в дампе.

6. Как секрет утекает на диск

Методичка требует, чтобы конфиденциальные данные не могли попасть на жёсткий диск. Путей туда три, и все три надо закрывать отдельно.

6.1. Раздел подкачки

Когда оперативной памяти не хватает, операционная система выгружает редко используемые страницы памяти на диск — в **раздел подкачки** (swap). Программа об этом не знает и никак не участвует. В результате страница с секретом оказывается в файле на диске и может пережить не только процесс, но и перезагрузку.

Защита — системный вызов `mlock`: он «прибивает» страницы памяти, запрещая их выгрузку. Применяется к небольшому буферу с секретами, потому что объём блокируемой памяти ограничен.

6.2. Дамп при падении

При аварийном завершении ядро сохраняет снимок памяти на диск. Защита — установить нулевой предел на размер такого файла (`RLIMIT_CORE`), тогда дамп не создаётся вовсе.

6.3. Снятие дампа у работающего процесса

Отладчик может присоединиться к работающему процессу и выгрузить его память. Механизм называется **ptrace**. Защита — флаг `PR_SET_DUMPABLE`: процесс сам объявляет, что его нельзя ни отлаживать, ни дампит. После этого системные файлы процесса переходят во владение `root`, и обычный пользователь до памяти не доберётся.

ПРО ОГРАНИЧЕНИЕ PTRACE В СОВРЕМЕННЫХ СИСТЕМАХ

В Linux есть дополнительный механизм защиты (yama), который по умолчанию разрешает отлаживать только **собственные дочерние** процессы. Снять дамп с постороннего процесса, даже своего собственного пользователя, без прав администратора нельзя.

Это подтверждено экспериментом и влияет на способ выполнения работы: требуемые дампы будут сниматься либо самим процессом (это разрешено всегда), либо в контейнере с явно выданной возможностью отладки.

7. Средства защиты памяти

Сводка всех мер. Каждая из них проверена на работоспособность в подготовительных экспериментах.

Мера	Что делает	От чего защищает
Хранение в <code>bytearray</code>	даёт возможность перезаписать содержимое на месте	от того, что секрет останется в памяти навсегда

Затирание нулями (<code>memset</code>)	перезаписывает буфер после использования	от чтения секрета из дампа после работы с ним
Строгий путь ввода	чтение сразу в изменяемый буфер, разбор без копий	от неудаляемых промежуточных копий — главная мера
<code>mlock</code>	запрещает выгрузку страниц памяти на диск	от попадания секрета в раздел подкачки
<code>RLIMIT_CORE = 0</code>	запрещает создание дампа при падении	от секрета в файле дампа на диске
<code>PR_SET_DUMPABLE = 0</code>	запрещает отладку и снятие дампа процесса	от снятия памяти у работающего процесса
Шифрование сразу после ввода	в памяти остаётся только шифротекст	от чтения данных, даже если дмп всё-таки снят
Хеширование вместо шифрования	исходное значение не хранится вовсе	полностью — но значение уже не восстановить

Шифрование или хеширование — что выбрать здесь

Задание допускает и то, и другое: «выполнить шифрование или хеширование этих данных». Выбор зависит от того, нужно ли потом показать значение пользователю.

В планируемом приложении реализуются **оба варианта** плюс наивный, и это не избыточность, а инструмент исследования: три режима позволяют сравнить результаты в одинаковых условиях.

8. Профилирование: память и процессорное время

Третий пункт задания требует проанализировать оперативную память и процессорное время приложения. Простыми словами — измерить, сколько ресурсов программа потребляет и на что их тратит.

Средство	Что показывает
<code>tracemalloc</code>	сколько памяти выделено и какой строкой кода . Умеет делать снимки и показывать разницу между ними — идеально для трёх требуемых точек: после ввода, после обновления, после удаления
<code>cProfile</code>	сколько раз вызвана каждая функция и сколько процессорного времени в ней проведено. Названа в методичке прямо
<code>getrusage</code>	суммарные показатели процесса: пиковое потребление памяти, время в пользовательском режиме и в режиме ядра
Файл состояния процесса в <code>/proc</code>	текущее потребление памяти, сколько выгружено в подкачку, сколько страниц заблокировано от выгрузки

ЧТО ИНТЕРЕСНО БУДЕТ УВИДЕТЬ

Argon2id по построению требует 64 МиБ памяти на одну операцию. На графике потребления это будет отчётливый пик. Отсюда прямая перекличка со второй работой: там из этого свойства следовало, что форма входа сама является ресурсом и лимит частоты запросов должен стоять до вычисления хеша. В третьей работе это станет видно измерением, а не рассуждением.

Второе интересное измерение — `VmSwap` и `VmLck`: они покажут, что заблокированные страницы действительно не уходят в подкачку.

9. Результаты подготовительных экспериментов

Одиннадцать экспериментов выполнены на целевой машине до начала реализации, чтобы проект опирался на измерения, а не на предположения.

Эксперимент	Результат
Снятие дампа с постороннего процесса штатной утилитой	Заблокировано механизмом уата: отлаживать разрешено только дочерние процессы
Самодамп: процесс читает собственную память	Работает без привилегий , размер дампа 24-32 МБ, содержимое ищется обычным поиском по подстроке
Секрет в неизменяемой строке	4 вхождения в дампе, затереть невозможно
<code>del</code> плюс явный вызов сборщика мусора	4 вхождения — не помогает: память освобождена, но не обнулена
Изменяемый буфер с затиранием нулями	Убирает ровно затёртую копию (было 4, стало 3)

Строгий путь: чтение в изменяемый буфер, разбор без копий, затираание	0 вхождений — цель достижима
Поиск источника оставшихся вхождений	Входной путь, а не хранение: стандартное чтение строки возвращает неизменяемые данные, преобразование и разбиение создают неудаляемые копии
Блокировка страниц памяти от выгрузки	Успешно без прав администратора ; подтверждено значениями «заблокировано 8 КБ», «выгружено 0»
Запрет создания дампа при падении	Предел устанавливается из самого процесса
Запрет отладки и снятия дампа	Самодамп заблокирован , системные файлы процесса переходят во владение <code>root</code> — внешний доступ тоже закрыт
Наличие раздела подкачки на машине	Присутствует, 4 ГиБ — значит требование «секреты не должны попадать на диск» проверяемо на практике

ДВА ВЫВОДА, ОПРЕДЕЛЯЮЩИХ ВСЮ РАБОТУ

Первый: есть достижимый положительный результат. Строгий путь на изменяемых буферах даёт ноль вхождений секрета в дампе. Значит в отчёте будет измеримая победа, а не констатация «язык так не умеет».

Второй: главная находка не там, где её ждут. Затереть своё хранилище легко — невозможно затереть промежуточные копии на пути ввода. Это содержательный результат части 3.1 и естественное ядро методики части 3.2: первым пунктом методики станет проверка не хранения, а именно пути следования секрета.

10. Что предстоит реализовать

10.1. Замысел приложения

`pziis-memvault` — хранилище записей целиком в оперативной памяти. Два класса данных, как во второй работе: неконфиденциальные хранятся открыто, конфиденциальные обрабатываются одним из трёх режимов.

Режим	Что делает	Ожидаемый результат в дампе
NAIVE	секрет в обычной строке, хранится как есть	несколько вхождений, не убираются
HASHED	только необратимый отпечаток Argon2id	открытого текста нет, но восстановить значение нельзя
SEALED	шифрование AES-256-GCM; открытый текст только в изменяемом буфере, затирается сразу после шифрования	ноль вхождений

Почему три режима, а не один. Одни и те же операции, одни и те же точки снятия дампа, разная обработка — это превращает отчёт из описания в **эксперимент с контрольной группой**. Измеряется не «защищено ли», а насколько и за счёт чего.

Итоговая таблица работы будет такой, и в её клетках — число вхождений секрета и области памяти, где он найден:

Режим	После ввода	После обновления	После удаления
NAIVE	?	?	?
HASHED	?	?	?
SEALED	?	?	?

10.2. План по этапам

Этап	Что делается	Результат
0	организационное	согласовать, что будет «предоставленным алгоритмом» в части 3.2, и допустим ли Python
1	скелет приложения	модель данных в памяти, меню, операции создания, обновления, удаления и просмотра — без криптографии
2	жизненный цикл секрета	строгий путь ввода на изменяемых буферах, три режима, затирание
3	подсистема дампов	самодамп, внешний дамп в контейнере, сканер с раскладкой вхождений по областям памяти, автоснимок в трёх точках
4	измерения	снимки потребления памяти, профиль процессорного времени, сводные показатели процесса
5	ужесточение	блокировка страниц, запрет дампов, запрет отладки, безопасный обработчик исключений

6	анализ кода	матрица невалидного ввода, поведение при сбое, статический анализ, фаззинг разборщика ввода
7	сбор данных	прогон трёх режимов в трёх точках, итоговая таблица, графики
8	отчёт 3.1	шесть разделов по методичке
9	методика	критерии со шкалой и весами
10	применение методики	аудит предоставленного варианта, сравнительная таблица, отчёт 3.2

10.3. Привязка к перечню CWE

CWE	Название
316	Открытое хранение конфиденциальных данных в памяти
226	Данные не удалены перед освобождением ресурса
244	Некорректная очистка кучи перед освобождением
528	Доступность соге-дампа посторонним
591	Секреты в незаблокированной памяти
20	Некорректная валидация ввода
400	Неконтролируемое потребление ресурсов
209	Утечка информации через сообщение об ошибке
532	Утечка конфиденциальных данных через журналы

11. Часть 3.2: методика оценки

Задание части 3.2 короткое: на основе результатов части 3.1 создать **методику оценки безопасности кода** и по ней сравнить предоставленный алгоритм со своим.

Что такое методика в этом контексте

Не разбор одной программы, а **воспроизводимая процедура**: набор критериев, по которым можно оценить любой код, со шкалой и правилами выставления оценки. Проверка «методика ли это» простая: если по документу другой человек оценит тот же код и получит тот же результат — методика; если получится другое — это было эссе.

Предполагаемые группы критериев

№	Группа	Что проверяется
1	Валидация ввода	длины, типы, диапазоны, управляющие символы, нормализация; наличие валидационных сообщений вместо падения
2	Обработка ошибок	не раскрывают ли сообщения и трассировки конфиденциальные данные
3	Жизненный цикл секрета в памяти	путь от ввода до хранения, наличие неизменяемых промежуточных копий, затирание после использования — главный критерий, вытекающий из находки части 3.1
4	Стойкость к снятию дампа	блокировка страниц, запрет дампов, запрет отладки
5	Ограничение ресурсов	поведение под нагрузкой, пределы на размер ввода
6	Гигиена зависимостей	фиксация версий, известные уязвимости
7	Покрывание проверками	проверяются ли свойства безопасности автоматически

Шкала обязательна. Без числовой оценки по каждому критерию «сравнение» окажется описательным, а методичка требует именно сравнения. Простейший работающий вариант: 0 — не выполнено, 1 — частично, 2 — выполнено, с весами по важности групп.

ЧТО УЖЕ ГОТОВО ДЛЯ ЧАСТИ 3.2

Одиннадцать проведённых экспериментов — это готовая основа критерия № 3, причём с измеримыми ожиданиями: известно, что наивное хранение даёт несколько вхождений, а строгий путь — ноль. Значит критерий можно сформулировать не как «аккуратно ли работают с памятью», а как проверяемое условие с числовым результатом.

12. Вопросы и ответы

В. Что такое безопасный код?

О. По определению методички — код, минимизирующий риски нарушения конфиденциальности и доступности данных **в любых ситуациях**. Учитывать надо три: невалидные входные данные, аварийные ситуации и большие нагрузки.

В. Чем третья работа отличается от второй?

О. Вторая смотрела на систему снаружи — по принципу чёрного ящика. Третья открывает код и оперативную память, то есть работает как белый ящик. И объект другой: во второй данные лежали в базе, в третьей — только в памяти.

В. Почему в третьей работе нет базы данных?

О. Потому что задание требует создавать, обновлять и удалять данные в оперативной памяти. Предмет изучения — не доступ к данным, а их жизненный цикл внутри процесса: что осталось в памяти после того, как с секретом поработали.

В. Что такое дамп памяти и откуда он берётся?

О. Побайтовая копия памяти процесса в файле. Создаётся ядром при падении программы, отладчиком у работающего процесса или средствами криминалистического анализа для всей машины. Опасен тем, что оказывается обычным файлом на диске и может попасть в резервную копию или в систему сбора отчётов об ошибках.

В. Почему нельзя просто удалить переменную с секретом?

О. Удаление и сборка мусора не стирают данные, а лишь помечают память свободной. Содержимое лежит там, пока его случайно не перезапишут. В эксперименте после удаления и явного вызова сборщика секрет находился в дампе столько же раз, сколько и до удаления.

В. Почему строку нельзя затереть, а `bytearray` можно?

О. Строка — неизменяемый объект: изменить её на месте невозможно, можно лишь создать новую, а старая останется в памяти. `bytearray` изменяемый, поэтому его содержимое можно перезаписать нулями по тому же адресу.

В. Что оказалось главной причиной утечки?

О. Не способ хранения, а **путь ввода**. Пока секрет идёт от ввода до буфера, создаются промежуточные копии, и все они неизменяемые: стандартное чтение строки возвращает неизменяемые данные, преобразование типа и разбиение на части добавляют ещё копии. Затереть их нельзя. Поэтому секрет должен жить в изменяемых буферах от начала и до конца.

В. Так можно ли вообще добиться нуля в Python?

О. Да, и это проверено: при чтении сразу в заранее выделенный изменяемый буфер, разборе без создания копий и затирании после использования секрет в дампе не находится ни разу.

В. Что такое `mlock` и зачем он?

О. Системный вызов, запрещающий выгрузку страниц памяти в раздел подкачки. Без него операционная система может сама, без участия программы, записать страницу с секретом на диск, и она переживёт даже перезагрузку. Применяется к небольшому буферу, потому что объём блокируемой памяти ограничен.

В. Как запретить снятие дампа с работающего процесса?

О. Флагом `PR_SET_DUMPABLE`: процесс объявляет, что его нельзя отлаживать и дампить. Проверено — после этого блокируется даже чтение собственной памяти, а системные файлы процесса переходят во владение `root`, поэтому обычный пользователь не подберётся и снаружи.

В. Шифровать или хешировать данные в памяти?

О. Зависит от того, надо ли потом показать значение. Если надо — шифровать, в памяти останется шифротекст. Если достаточно проверять совпадение — хешировать, тогда исходного значения нет вовсе и защита полная. В приложении планируются оба варианта плюс наивный, чтобы сравнить их в одинаковых условиях.

В. Зачем три режима, а не сразу самый защищённый?

О. Чтобы получилось измерение, а не утверждение. Одни и те же операции и точки снятия дампа при разной обработке дают контрольную группу: видно, сколько вхождений даёт наивный вариант и сколько защищённый. Без наивного режима не с чем сравнивать.

В. Почему требуется именно три дампа?

О. Так задано в методичке: после ввода данных, после обновления и после удаления. Смысл в том, чтобы проследить жизненный цикл: появление секрета, его замену при обновлении (осталась ли прежняя версия) и то, что от него осталось после удаления.

В. Чем анализируют память и процессорное время?

О. Потребление памяти с привязкой к строкам кода — `tracemalloc`, он умеет делать снимки и показывать разницу, что удобно для трёх требуемых точек. Процессорное время — `cProfile`, названный в методичке. Сводные показатели процесса и объём выгруженного в подкачку — через `getrusage` и файл состояния процесса.

В. Что такое валидационные сообщения?

О. Подход, при котором программа на неверный ввод не падает и не портит данные, а объясняет пользователю, что именно неправильно. Проверять надо по белому списку: перечислять допустимое, потому что недопустимого всегда больше, чем можно предусмотреть.

В. Что должна содержать методика в части 3.2?

О. Не разбор одной программы, а воспроизводимую процедуру: критерии, шкалу и правила выставления оценки, применимые к любому коду. Проверка простая: если по документу другой человек оценит тот же код и получит тот же результат — это методика.

В. Почему нельзя снять дампы с чужого процесса без прав администратора?

О. Из-за механизма уама в Linux: по умолчанию отлаживать разрешено только собственные дочерние процессы. Поэтому дампы в работе снимаются либо самим процессом — это разрешено всегда, — либо в контейнере с явно выданной возможностью отладки.

13. Итог

Что уже сделано. Разобраны обе части задания, проведены одиннадцать подготовительных экспериментов на целевой машине, выбран стек и форма интерфейса, составлен план из одиннадцати этапов, зафиксирована привязка к перечню CWE. Приложение пока не реализовано — это предмет этапов с первого по седьмой.

Семантическая составляющая (предварительно). Работа показывает, что защита данных в оперативной памяти отличается от защиты данных в хранилище принципиально. Хранилище можно зашифровать целиком, а в памяти секрет обязан когда-то побывать в открытом виде — иначе с ним невозможно работать. Поэтому предметом защиты становится не сам факт наличия секрета в памяти, а **длительность и количество его копий**. И главная трудность обнаруживается не там, где её ожидают: аккуратно затереть своё хранилище просто, а вот путь, которым секрет попадает в программу, состоит из неизменяемых промежуточных копий, ни одну из которых стереть нельзя. Отсюда практический вывод: безопасность работы с секретом определяется конструкцией **всего пути** данных, а не одной точкой хранения.

Прагматическая составляющая (предварительно). Установлено, что цель достижима: строгий путь на изменяемых буферах даёт ноль вхождений секрета в дампы, и это измеримый результат, а не рассуждение. Проверена работоспособность всей цепочки защиты — блокировка страниц от выгрузки, запрет дампа при падении, запрет отладки процесса, — причём без прав администратора. Найдено ограничение окружения (запрет отладки посторонних процессов) и найден обход, пригодный для воспроизводимого снятия трёх требуемых дампов. Полученные измерения сразу становятся основой главного критерия методики части 3.2, что делает её не описательной, а числовой.